

Clusters and Compute

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering cluster startup, autoscaling, driver and worker sizing, spot instances, cluster policies, instance pools, and runtime configuration in Databricks. Patterns every Databricks engineer must recognise.

INTRODUCTION

Compute is not just infrastructure. It is your first design decision.

Most Azure Databricks performance problems are not code problems. They are compute problems. The wrong cluster type, the wrong VM SKU, autoscaling configured without understanding how it works, Spot VMs without a fallback strategy, workspaces that span regions while their storage lives elsewhere. These decisions are made once during setup and then forgotten until something breaks at scale.

This book covers seven chapters. Chapter 1 covers cluster startup and cold start patterns specific to Azure VM allocation, VNet injection, and secure cluster connectivity. Chapter 2 covers autoscaling and the ways it can make jobs slower rather than faster. Chapter 3 covers driver and worker sizing decisions across Azure VM SKU families and the consequences of getting them wrong. Chapter 4 covers Azure Spot VMs and the failure modes that appear when eviction happens at the worst moment. Chapter 5 covers cluster policies and governance. Chapter 6 covers instance pools, including the dual Azure VM and DBU billing model that catches teams by surprise. Chapter 7 covers Databricks Runtime and configuration and the subtle differences between runtime versions that cause silent failures.

Every scenario follows the same structure as the rest of the series: the situation, the wrong thinking, what is actually happening, and the fix. The wrong thinking section is the most important part. Understanding why the wrong answer was reasonable is what builds the intuition to avoid the next problem before it appears.

01

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 1 Cluster Startup

Eight scenarios covering cold start latency, the hidden cost of job cluster provisioning, and the specific situations where startup time dominates total job duration on Azure.

01

CLUSTER STARTUP

Your job failed in 30 seconds. The cluster spent 28 of them starting up.

THE SITUATION

An Azure Databricks job runs on a job cluster and executes a notebook that completes in under 2 seconds when the cluster is warm. In production the job consistently takes 30 seconds total and occasionally fails due to a downstream timeout expecting completion in under 20 seconds.

WRONG THINKING

Engineers check the Spark UI and see 2 seconds of execution time. They assume the timeout is too aggressive and increase it. The Spark UI only shows execution time after the cluster is running, not total wall clock time including startup. The root cause is never identified.

WHAT IS ACTUALLY HAPPENING

Job clusters in Azure Databricks provision fresh Azure VMs for every run. Provisioning involves requesting VMs from Azure Compute, attaching them to the workspace VNet (or the managed VNet), pulling and installing the Databricks runtime image, configuring networking with secure cluster connectivity, and starting the Spark session. On classic compute this typically takes 4 to 7 minutes for general-purpose `Standard_DS` or `Standard_D_v5` family VMs, and longer for memory-optimised `Standard_E` SKUs at peak hours. For a job that executes in 2 seconds, startup overhead dominates total duration. The Spark UI shows only the execution phase. The startup phase is visible in the cluster event log but not the job run summary.

THE FIX

For short-running jobs where startup cost dominates, switch to serverless compute which starts in seconds. For jobs that must use classic compute, attach them to an instance pool with pre-warmed VMs. Alternatively chain multiple short tasks into a single job so startup cost is paid once across all tasks. Always measure total wall clock time from job submission to completion, not just Spark execution time, when evaluating job performance.

02

CLUSTER STARTUP

You scheduled a job every 5 minutes. A new cluster spins up every time.

THE SITUATION

An Azure Databricks job runs every 5 minutes to process incoming data. Each run takes 90 seconds of actual processing. Total time from job trigger to completion is consistently 7 to 9 minutes because a new job cluster is provisioned for every run and fully terminated between runs.

WRONG THINKING

Engineers set up the schedule expecting a 90-second job to run comfortably in a 5-minute window. The cluster configuration was copied from another job and not reviewed. Auto-termination after each run seemed like good cost management. The interaction between auto-termination and schedule frequency was not considered.

WHAT IS ACTUALLY HAPPENING

Job clusters terminate after each run by default. When the next scheduled run fires 5 minutes later, a new cluster must be provisioned from scratch. On Azure that means a new VM allocation request to Azure Compute, possibly subject to subscription quota and regional capacity at the time of the request. If provisioning takes 5 to 7 minutes and the schedule interval is 5 minutes, the cluster for the next run is not ready before the previous one has barely finished. Each run waits for cluster provisioning, making a 90-second job take 7 to 9 minutes end to end. The schedule and cluster type are fundamentally mismatched.

THE FIX

For high-frequency jobs switch to serverless compute which provisions in seconds and eliminates the mismatch between schedule interval and cluster startup time. If classic compute is required, use a continuous job with a trigger interval rather than a scheduled job with a new cluster per run. For jobs that must use dedicated clusters, set the schedule interval to be significantly longer than startup time plus execution time.

03

CLUSTER STARTUP

Your all-purpose cluster is always warm. Your job cluster starts cold every run.

THE SITUATION

A data engineer develops notebooks on an always-running all-purpose cluster where execution takes seconds. When promoted to production as job cluster runs, the same notebooks jump from seconds to minutes. The code has not changed.

WRONG THINKING

Engineers assume cluster type is an administrative distinction with no performance impact. Since the notebook ran in 2 seconds on the all-purpose cluster they expect the same on a job cluster. Cold start is never encountered during development because the development cluster is always warm.

WHAT IS ACTUALLY HAPPENING

An always-running all-purpose cluster has already completed startup. Notebooks connecting to it skip directly to execution. A job cluster provisions fresh Azure VMs for every run and the first task always pays the full cold start cost: VM allocation, image pull, VNet attachment, and Spark session initialisation. Development on an always-on cluster hides this cost completely because it is never incurred during development. Production reveals it.

THE FIX

Never use an all-purpose cluster as the performance baseline for job cluster workloads. Test job performance using a job cluster or serverless compute before promoting to production. For latency-sensitive jobs use serverless compute or instance pools to reduce cold start time. Document the expected startup overhead for each job cluster configuration so downstream SLAs are set against total wall clock time.

04

CLUSTER STARTUP

You increased cluster size expecting faster startup. It got slower.

THE SITUATION

A job cluster with 4 worker nodes starts in approximately 4 minutes. A data engineer increases it to 20 workers expecting faster startup and faster execution. Startup time increases to 7 to 9 minutes and occasionally fails with a quota or capacity error.

WRONG THINKING

Engineers assume larger clusters start faster because they have more compute. More nodes should mean more parallelism and speed in all phases including startup.

WHAT IS ACTUALLY HAPPENING

On Azure, cluster startup time does not scale linearly with node count because requesting many VMs at once interacts with two limits that small clusters never hit. The first is subscription and region quota for the specific VM SKU: requesting 20 `Standard_DS4_v2` instances may exceed the per-region vCPU quota and Azure throttles the request, processing it in batches. The second is availability zone capacity: at peak hours a popular SKU may have limited capacity in the workspace's primary zone, causing some VM requests to retry or fail. Azure does parallelise VM allocation, but quota throttling and zone capacity dominate startup time for large clusters in busy regions.

THE FIX

Right-size clusters for the actual workload rather than provisioning large clusters to feel faster. For startup-sensitive jobs prefer fewer larger VM SKUs over many smaller ones, for example 4 `Standard_E16ds_v5` instances rather than 20 `Standard_DS3_v2` instances. Check vCPU quotas for the target SKU and region in the Azure portal before scaling up. Use serverless compute for jobs where startup latency is the primary concern.

05

CLUSTER STARTUP

Your cluster shows Running. Your notebook still hangs on attach.

THE SITUATION

A cluster reaches Running status in the Azure Databricks UI. A data engineer attaches a notebook and runs a cell. The cell hangs for 2 to 3 minutes before executing. This happens consistently on the first cell after a cluster start regardless of cell complexity.

WRONG THINKING

Engineers assume Running status means the cluster is fully ready for work. When the notebook hangs after showing Running, they assume a notebook or connectivity issue and restart the notebook or cluster.

WHAT IS ACTUALLY HAPPENING

Running status indicates VMs are provisioned and cluster infrastructure is up. It does not mean the Spark session is fully initialised. After VMs reach running state, Azure Databricks still needs to start Spark master and worker processes, configure executor JVMs, load init scripts (often from DBFS or ADLS Gen2 with abfss), install libraries specified in the cluster configuration, and establish connections between driver and all worker nodes. If the workspace uses VNet injection, additional private endpoint or NSG rule evaluation can add to this phase. This post-provisioning phase can take 1 to 3 minutes after the cluster shows Running.

THE FIX

Wait for the Spark context to be initialised before sending the first command. Check the Spark UI link rather than relying on the Running status indicator. For automated workflows, use the Databricks REST API to poll for Spark context availability rather than triggering the first task immediately after the cluster reaches Running status.

06

CLUSTER STARTUP

You terminated a cluster to save cost. The next job waited 8 minutes to start.

THE SITUATION

A data engineer manually terminates an all-purpose cluster at the end of the working day. An automated job that runs at 7am and was using this cluster waits 8 minutes before processing begins. Business users notice the data is 10 minutes late.

WRONG THINKING

Engineers assume manually terminating idle clusters is always the right cost-saving action. Downstream jobs that depend on a warm cluster are not considered when making the termination decision.

WHAT IS ACTUALLY HAPPENING

When a cluster is manually terminated, any subsequent job must provision a new one from scratch. On Azure that means new VM allocation requests subject to quota and regional capacity at 7am, an image pull, init script execution, and library installation. An 8-minute startup overhead is added to the first job run after termination. For business-critical morning jobs where data freshness is measured in minutes, this startup delay can violate SLAs. The cost saved by terminating the cluster overnight may be smaller than the business impact of late morning data delivery.

THE FIX

For clusters that serve time-sensitive morning jobs, configure auto-start to bring the cluster online before the first scheduled job fires. Use a pre-warm job that runs 10 minutes before the first data job. Alternatively switch morning jobs to serverless compute which eliminates the warm cluster dependency entirely. Evaluate the cost of startup delay against the cost of idle overnight compute before deciding to terminate clusters that serve morning SLAs.

07

CLUSTER STARTUP

Your cluster starts in dev in 2 minutes. Takes 6 minutes in production.

THE SITUATION

A job cluster provisions in approximately 2 minutes in the development workspace but takes 6 minutes with the same configuration in the production workspace. VM SKU, worker count, and runtime version are all identical.

WRONG THINKING

Engineers assume the same cluster configuration produces the same startup time in all workspaces. When production startup is 3 times slower, an Azure Databricks platform issue is suspected and a support ticket is raised. Differences in workspace networking and Azure region capacity are not considered.

WHAT IS ACTUALLY HAPPENING

Azure Databricks workspaces are deployed to specific Azure regions and increasingly use VNet injection or managed VNet with secure cluster connectivity. Development and production workspaces are often in different regions, different subscriptions, or use different networking models. VM availability for a given SKU, ExpressRoute or private endpoint latency, NSG rule evaluation, and Azure region capacity all vary between deployments. A workspace with VNet injection, private endpoints to ADLS Gen2, and no public IP will consistently start clusters slower than a workspace using the managed VNet because each VM must complete additional networking setup before Spark can initialise.

THE FIX

Always measure cluster startup times in the target workspace and region rather than assuming dev times translate to production. If production startup is significantly longer, check whether the workspace uses VNet injection with private endpoints, and whether the target VM SKU has high allocation rates in that region. Consider switching to a SKU with better regional availability or enabling an instance pool to absorb the steady-state cost. Monitor cluster startup times over time as Azure region capacity varies.

08

CLUSTER STARTUP

You reused an all-purpose cluster for jobs. Cost doubled unexpectedly.

THE SITUATION

To avoid job cluster cold start delays a team runs all jobs on a shared all-purpose cluster kept running continuously. Cold start delays are eliminated. The monthly Azure Databricks bill doubles compared to the previous month when jobs used dedicated job clusters.

WRONG THINKING

Engineers focus on eliminating startup latency and succeed. Cost is evaluated after the fact. The DBU pricing difference between all-purpose and job compute is not considered when making the architectural decision.

WHAT IS ACTUALLY HAPPENING

Azure Databricks charges different DBU rates for all-purpose compute and job compute. All-purpose compute DBUs are priced approximately 2 to 3 times higher than job compute DBUs on Azure depending on the workspace tier (Standard or Premium). On top of DBU charges, the underlying Azure VM cost (visible in the Azure consumption report, separate from the Databricks marketplace charge) accrues continuously while the cluster runs. A cluster running as all-purpose compute for 24 hours costs substantially more than the equivalent job clusters that only run during actual job execution. The cold start problem is real but keeping a large all-purpose cluster warm continuously often costs more than the productivity impact of the cold starts it eliminates.

THE FIX

Calculate total cost (DBUs plus Azure VM cost) of a continuously running all-purpose cluster versus equivalent job compute with startup overhead before making architectural decisions. For most production workloads, job compute or serverless compute is significantly cheaper than all-purpose compute even accounting for startup time. Reserve all-purpose compute for interactive development work. For production jobs where startup latency must be minimised, use instance pools with job compute rather than switching to all-purpose compute. Consider Azure Reserved Instances or Savings Plans for the underlying VMs to reduce steady-state VM cost by 30 to 60 percent.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 2

Autoscaling

Seven scenarios covering how Databricks autoscaling works on Azure, why it sometimes makes jobs slower rather than faster, and the specific workload patterns where autoscaling creates more problems than it solves.

09

AUTOSCALING

You enabled autoscaling. Your job got slower not faster.

THE SITUATION

A Databricks job running on a fixed 8-node cluster takes 45 minutes. Autoscaling is enabled with min 2 and max 20 nodes expecting the cluster to scale up and process the job faster. After enabling autoscaling the same job takes 65 minutes.

WRONG THINKING

Engineers assume autoscaling always improves performance by adding resources dynamically. More nodes should mean faster processing. The overhead of scaling operations themselves is not considered.

WHAT IS ACTUALLY HAPPENING

Autoscaling adds and removes nodes based on observed demand. The scaling operation itself takes time: new Azure VMs must be allocated (subject to quota and zone capacity), added to the cluster, and registered with the Spark scheduler before they can receive tasks. For a job with a single large shuffle stage the scaling event may fire partway through the stage. Newly added nodes cannot participate in the current shuffle and the cluster pays provisioning overhead without receiving performance benefit until the next stage. For jobs completing in under an hour with few large stages, autoscaling overhead frequently exceeds the benefit.

THE FIX

Evaluate whether a workload benefits from autoscaling before enabling it. Autoscaling helps most for jobs with highly variable parallelism across stages, long-running jobs with multiple distinct processing phases, and interactive workloads with unpredictable demand. For batch jobs with predictable data volumes and uniform stage sizes a correctly sized fixed cluster outperforms autoscaling. Use the Spark UI to identify the stage structure before deciding whether autoscaling is appropriate.

10

AUTOSCALING

Your cluster scaled up to 20 workers. Only 2 were actually doing work.

THE SITUATION

An autoscaling cluster scales up to its maximum of 20 workers during a job run. The Spark UI shows 18 workers idle while 2 workers process all tasks. The job takes as long as on a 2-node cluster and costs 10 times as much.

WRONG THINKING

Engineers assume that if the cluster scaled up the additional nodes must be processing something. The autoscaling algorithm is trusted to add nodes only when needed. The Spark UI is not checked to verify all nodes are active.

WHAT IS ACTUALLY HAPPENING

Autoscaling fires based on task queue depth and executor utilisation. If a job has data skew where one or two partitions are significantly larger than others, those large partitions occupy 1 or 2 executors for most of the job while all other executors finish their small partitions quickly and become idle. The autoscaler sees high utilisation on those 2 executors and scales up, adding 18 more workers that immediately become idle because there is no work to give them. The cluster scaled up in response to a skew problem, not a resource scarcity problem. More workers cannot fix skew.

THE FIX

Before enabling autoscaling, profile the job's data distribution to identify skew. Check the Spark UI task duration distribution within a stage. If maximum task duration is significantly longer than the median, skew is the bottleneck not resource scarcity. Fix skew using salting, repartitioning, or AQE skew hints before adding autoscaling. A well-distributed job on a fixed cluster will outperform an autoscaled job with skewed data.

11

AUTOSCALING

Autoscaling fired mid-job. Shuffle data was lost and the job restarted from scratch.

THE SITUATION

A job running on a classic compute cluster with autoscaling enabled fails partway through a large shuffle stage. The error references missing shuffle files. The job retries from the beginning rather than from the failed stage.

WRONG THINKING

Engineers assume Databricks autoscaling is always safe to use with any workload. The interaction between scale-down events and shuffle file storage on local VM disks is not known. Autoscaling is treated as a simple on-off feature without understanding the underlying behavior.

WHAT IS ACTUALLY HAPPENING

On Azure Databricks Premium tier and above, autoscaling is shuffle-aware (optimized autoscaling): it will not remove a node that holds shuffle data needed by a future stage, and on job compute it scales down after 40 seconds of underutilisation while on all-purpose compute it waits 150 seconds. On the Standard pricing tier the workspace uses standard autoscaling, which is more aggressive about decommissioning workers during job execution and is not shuffle-aware. When a worker is removed, shuffle files stored on its local VM temp disk are lost. If a subsequent stage needs those shuffle files the stage fails because the files are no longer accessible. This forces a retry of the failed stage or a full job restart.

THE FIX

For shuffle-heavy batch workloads use jobs compute on a Premium tier (or above) workspace, which gives you optimized shuffle-aware autoscaling. Avoid running shuffle-heavy jobs on autoscaling all-purpose clusters because their longer scale-down window does not eliminate the risk on Standard tier. For workloads where autoscaling behavior cannot be controlled, use a fixed cluster size for shuffle-heavy stages. Alternatively switch shuffle-heavy batch workloads to serverless compute which uses disaggregated shuffle and handles resource management without exposing shuffle file dependencies to scale-down operations.

12

AUTOSCALING

You set min 2 max 10 workers. The cluster never scaled above 2.

THE SITUATION

An autoscaling cluster is configured with min 2 max 10 workers. A data engineer expects it to scale up during heavy processing. After weeks of monitoring the cluster has never scaled above 2 workers. Jobs are slow but no scaling events appear in the event log.

WRONG THINKING

Engineers assume autoscaling will scale up when jobs run slowly. Slow performance is attributed to other causes. The autoscaling trigger conditions are not investigated because scaling up seems like it should happen automatically when any job is running.

WHAT IS ACTUALLY HAPPENING

Databricks autoscaling scales up based on task queue depth. If there are pending tasks that cannot be scheduled because all executors are busy, the autoscaler adds workers. If the job has only 4 partitions and 2 workers each with 4 cores, all 8 available executor slots can be filled by the 4 tasks without any tasks queuing. The autoscaler sees no pending tasks and does not scale up even though the cluster is fully utilised. The bottleneck is partition count not executor count. Autoscaling cannot help when there are not enough partitions to drive parallelism.

THE FIX

Check partition count before troubleshooting autoscaling. Use `df.rdd.getNumPartitions()` to verify. If partition count is less than or equal to total executor slots on the minimum cluster, autoscaling will never fire. Increase partition count using `repartition()` or by adjusting `spark.sql.shuffle.partitions` to a value exceeding maximum worker count times cores per worker. Only then will autoscaling have tasks to queue and a trigger condition to respond to.

13

AUTOSCALING

Your autoscaling cluster costs more than a fixed cluster doing the same work.

THE SITUATION

A team switches from a fixed 8-node cluster to an autoscaling cluster with min 4 max 12 nodes expecting to reduce costs by not paying for idle workers. After one month the autoscaling cluster costs 30 percent more than the fixed cluster did for the same workload.

WRONG THINKING

Engineers assume autoscaling always reduces cost by right-sizing compute to actual demand. The cost of keeping the minimum node count running continuously plus DBU and Azure VM overhead of scale-up and scale-down operations is not factored into the estimate.

WHAT IS ACTUALLY HAPPENING

Autoscaling reduces cost when the workload has significant idle periods and the cluster can scale down to the minimum. For workloads that run continuously or scale up to near-maximum frequently, autoscaling adds cost: the minimum node count runs continuously even during periods of no work, scale-up provisioning time means VMs are paid for before they are productive, and shuffle-aware autoscaling deliberately avoids aggressive scale-down to protect shuffle data leaving nodes running longer than necessary. On Azure, both DBU charges and VM charges accumulate during these idle and provisioning periods.

THE FIX

Model the expected cost of autoscaling before enabling it by analysing actual utilisation of the fixed cluster over time. If the fixed cluster runs above 70 percent utilisation for most of its active time, autoscaling will not save meaningful cost. Autoscaling delivers the greatest benefit for workloads with highly variable demand, long idle periods between bursts, or unpredictable data volumes. For steady-state batch workloads with predictable volumes a correctly sized fixed cluster is often both cheaper and more predictable, especially when combined with Azure Reserved Instances for the underlying VMs.

14

AUTOSCALING

Autoscaling added workers during the last 30 seconds of a 10 minute job.

THE SITUATION

An autoscaling cluster adds 8 new worker nodes during the final 30 seconds of a 10-minute job. The nodes provision and become available just as the job completes. The next job does not start for another 15 minutes. The 8 nodes sit idle for 15 minutes before being removed.

WRONG THINKING

Engineers assume autoscaling is smart enough to only add nodes when they will be useful. The provisioning lag between when the autoscaler decides to add nodes and when those nodes are actually available is not considered.

WHAT IS ACTUALLY HAPPENING

Autoscaling decisions are based on observed task queue depth and executor utilisation at the moment of evaluation. The algorithm cannot predict that a job is about to complete. If a final aggregation stage briefly queues tasks the autoscaler fires a scale-up request. Azure VM provisioning takes 3 to 5 minutes including quota validation and zone selection. By the time nodes are ready the job has completed and they are idle. The idle instance timeout then keeps them running before scale-down, generating cost with zero utilisation on both the DBU and VM lines of the bill.

THE FIX

Set the autoscaling idle timeout to a short value appropriate for your workload pattern to minimise cost of post-job idle nodes. For workloads where you can predict approximate job duration, consider disabling autoscaling and using a fixed cluster sized for expected peak. Monitor scale-up events in the cluster event log and correlate them with job stage timelines to identify whether scale-up events are occurring too late to benefit the triggering job.

15

AUTOSCALING

Your streaming job autoscaled to maximum workers and stayed there permanently.

THE SITUATION

A Structured Streaming job runs on an autoscaling cluster with min 2 max 20 workers. Shortly after the streaming job starts the cluster scales to 20 workers. It never scales back down even during periods of low input rate. The cluster runs at maximum cost continuously.

WRONG THINKING

Engineers assume autoscaling will scale down a streaming job during quiet periods the same way it scales down batch jobs. The streaming job is monitored for correctness but not for cluster utilisation. The cost of a permanently maximised streaming cluster is not flagged until a billing review.

WHAT IS ACTUALLY HAPPENING

Structured Streaming jobs maintain stateful operations and continuous micro-batch processing. The autoscaler evaluates executor utilisation across all active executors. A streaming job even during low-input-rate periods keeps executors busy processing micro-batches, maintaining watermarks, and managing state. Cluster utilisation rarely drops below the scale-down threshold because the streaming engine is always doing work even when input volume is low. Once the cluster scales up in response to a burst, the streaming workload prevents it from scaling back down. On Azure, this means both DBU and Azure VM costs stay pinned at the maximum tier indefinitely.

THE FIX

For streaming jobs set the minimum worker count to the size required to sustain expected steady-state input rate and the maximum to the size needed for peak bursts. Do not rely on autoscaling to manage streaming costs through scale-down. Consider using Delta Live Tables with enhanced autoscaling which has streaming-specific scale-down logic, or use serverless streaming compute. Monitor streaming cluster size over time and set Azure Cost Management alerts for clusters that stay at maximum for extended periods.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 3

Driver and Worker Sizing

Seven scenarios covering the consequences of getting driver and worker sizing wrong on Azure VM SKUs, why more workers is not always the answer, and where the bottleneck actually is when jobs run slowly.

16

DRIVER AND WORKER SIZING

Your job failed with out of memory on the driver. The workers were idle.

THE SITUATION

A Databricks job fails with a driver out of memory error. The worker nodes are showing less than 20 percent memory utilisation in the Spark UI. Adding more workers does not fix the problem. The job continues to fail at the same point.

WRONG THINKING

Engineers see an out of memory error and assume they need more memory somewhere. Adding more workers seems like the logical response. The distinction between driver memory and worker memory is not considered when diagnosing the failure.

WHAT IS ACTUALLY HAPPENING

The Spark driver is responsible for collecting results, broadcasting variables, managing the DAG, and running Python code that is not distributed. Operations like `collect()`, `toPandas()`, large Python dictionary operations, and broadcasting large lookup tables all consume driver memory, not worker memory. If the driver runs out of memory, adding more workers provides zero benefit because the bottleneck is on the single driver node. Worker memory is used for distributed Spark tasks. Driver memory is used for coordination and collection operations. They are independent resources.

THE FIX

Identify whether the OOM is on the driver or a worker executor by checking the error message and the Spark UI. If the OOM is on the driver, increase the driver VM SKU rather than adding workers; for example move from `Standard_DS4_v2` (28GB) to `Standard_E8ds_v5` (64GB). Review the job for operations that pull large datasets to the driver such as `collect()` on large DataFrames, large broadcast joins, or Python loops that accumulate data. Replace driver-side collection operations with distributed writes to ADLS Gen2 where possible. Set driver memory configuration explicitly using `spark.driver.memory` when the default is insufficient.

17

DRIVER AND WORKER SIZING

You increased worker count. The driver became the bottleneck.

THE SITUATION

A job is running slowly and a data engineer doubles the worker count from 8 to 16. The job runs at exactly the same speed. Increasing to 32 workers also produces no improvement. The cluster is visibly underutilised in the Spark UI with workers finishing tasks quickly and then waiting.

WRONG THINKING

Engineers assume that more workers always means faster jobs. The driver is never considered as a potential bottleneck because it is a single node and its activity is less visible than the distributed worker activity shown in the Spark UI.

WHAT IS ACTUALLY HAPPENING

When worker count is increased, workers complete their tasks faster and send results back to the driver more rapidly. The driver must process all of these results, coordinate the next stage, and manage the task scheduler. If the driver is CPU-bound or memory-bound managing the increased flow of results and task assignments, it becomes the limiting factor regardless of how many workers are available. The workers sit idle waiting for the driver to dispatch the next batch of tasks. This pattern is most common in jobs with thousands of small tasks, heavy shuffle coordination, or significant driver-side Python processing.

THE FIX

Monitor driver CPU and memory utilisation alongside worker utilisation when troubleshooting performance. If the driver is consistently at high CPU or memory while workers are idle, increase the driver VM SKU rather than adding workers. Move from a general-purpose D-series driver to a memory-optimised E-series driver, or to an F-series compute-optimised SKU if CPU is the bottleneck. Reduce the number of tasks if they are excessively small by coalescing partitions before shuffle operations. Avoid Python UDFs that run on the driver and replace them with Spark native functions or Pandas UDFs that run on workers.

18

DRIVER AND WORKER SIZING

Your collect() call worked in dev. Killed the driver in production.

THE SITUATION

A notebook that works correctly in development with a small dataset is promoted to production. The first production run fails with a driver out of memory error on a `collect()` call. The development dataset was 10,000 rows. The production dataset is 50 million rows.

WRONG THINKING

Engineers test with representative data volumes in development and assume the code will scale. A `collect()` call that works on 10,000 rows seems like a reasonable pattern. The memory implications of collecting 50 million rows to the driver are not considered during development.

WHAT IS ACTUALLY HAPPENING

`collect()` pulls the entire DataFrame from all workers to the driver node as a Python list. For a DataFrame with 50 million rows and 20 columns, this can require several gigabytes of driver memory depending on data types. The driver VM has a fixed memory allocation determined by its SKU, and if the collected data exceeds available driver memory the JVM process is killed. Development environments with small data volumes never reveal this because the collection fits in memory comfortably. Production reveals it immediately on first run.

THE FIX

Never use `collect()` on DataFrames of unknown or potentially large size in production code. Replace collection patterns with distributed writes using `write` to Delta tables on ADLS Gen2. If the result must be passed to downstream Python code, use `limit()` to cap the collection size, use `toLocalIterator()` to stream rows without materialising all of them in driver memory, or aggregate the data before collecting. Add an explicit row count check before any `collect()` call in production and fail with a clear error if the count exceeds a defined threshold.

19

DRIVER AND WORKER SIZING

You gave workers 64GB RAM each. Spark only used 30GB of it.

THE SITUATION

A cluster is provisioned with `Standard_E8ds_v5` workers giving each worker 64GB of advertised RAM. Spark UI memory metrics show workers using at most 30GB during the heaviest processing stages. The remaining 34GB appears unused. The cluster costs significantly more than a standard configuration.

WRONG THINKING

Engineers provision large memory instances to avoid OOM errors and ensure headroom. Since memory errors have caused problems before, more memory seems like a safe choice. The Spark memory model and Azure VM memory reservation overhead are not understood.

WHAT IS ACTUALLY HAPPENING

Two effects compound here. First, Azure VMs reserve memory for the hypervisor and the Azure Linux Agent: a SKU advertised as 64GB delivers closer to 56 to 58GB usable to the guest OS. Second, Spark does not use all available JVM memory for data processing. Available heap memory is divided into execution memory for shuffle and aggregation operations, storage memory for caching and broadcast variables, and reserved memory for internal Spark overhead. The effective fraction usable for data is roughly 50 to 55 percent of the executor heap. On a 64GB advertised worker, Spark may only effectively use 28 to 35GB for data operations. Provisioning `Standard_E8ds_v5` when `Standard_DS5_v2` would provide equivalent usable Spark memory wastes cost with no performance benefit.

THE FIX

Check actual Spark memory utilisation in the Spark UI under the Executors tab before sizing worker SKUs. The Storage Memory and Execution Memory columns show actual usage. If workers are consistently using less than 60 percent of their JVM heap, downsize to a smaller SKU. Note that the legacy parameters `spark.storage.memoryFraction` and `spark.shuffle.memoryFraction` were deprecated when the Unified Memory Manager was introduced in Spark 1.6 and should not be set; the modern setting is `spark.memory.fraction` which defaults to 0.6. Match VM memory to actual Spark workload requirements rather than provisioning for worst-case scenarios that rarely occur.

20

DRIVER AND WORKER SIZING

Your job runs faster on 4 large workers than 16 small ones.

THE SITUATION

A data engineer benchmarks two cluster configurations for a shuffle-heavy aggregation job. Configuration A uses 4 `Standard_E8ds_v5` workers with 8 cores and 64GB RAM each. Configuration B uses 16 `Standard_DS3_v2` workers with 2 cores and 8GB RAM each. Both configurations have the same total cores and comparable total memory. Configuration A is 40 percent faster and costs the same.

WRONG THINKING

Engineers assume total cores and total memory determine performance and that the distribution across workers is irrelevant. Configuration B has more workers which seems like it should provide more parallelism. The shuffle overhead of coordinating across 16 workers versus 4 is not considered.

WHAT IS ACTUALLY HAPPENING

Shuffle operations in Spark require data to be exchanged across all workers. The cost of a shuffle grows with the number of executors involved because each executor must send data to and receive data from every other executor. A shuffle across 16 small workers involves 256 network connections in the worst case. A shuffle across 4 large workers involves only 16. On Azure, accelerated networking on larger SKUs further reduces the per-connection latency compared to standard networking on smaller SKUs. The network overhead of coordinating 16 small workers dominates the job duration for shuffle-heavy workloads. Fewer larger workers reduce shuffle coordination overhead and benefit from better data locality within each worker's larger memory space.

THE FIX

For shuffle-heavy workloads prefer fewer larger workers over many small workers. Memory-optimised (E-series) or compute-optimised (F-series) SKUs with 8 to 16 cores each typically outperform clusters with the same total cores spread across many 2-core instances. Use the Spark UI shuffle read and write metrics to identify whether shuffle overhead is a significant fraction of total job time. If it is, consolidating workers onto larger SKUs is often the fastest optimisation available without changing the query logic.

21

DRIVER AND WORKER SIZING

The driver has 8 cores. Only 1 is ever being used.

THE SITUATION

A cluster uses a large `Standard_E8ds_v5` driver with 8 cores to handle complex coordination logic. Monitoring shows the driver CPU consistently at 12 to 15 percent utilisation, indicating roughly 1 of 8 cores is active. The expensive driver SKU is mostly idle.

WRONG THINKING

Engineers provision a large driver SKU to future-proof against driver bottlenecks experienced in the past. More cores on the driver should provide capacity for concurrent driver operations. The single-threaded nature of most Spark driver operations is not considered.

WHAT IS ACTUALLY HAPPENING

The Spark driver runs most of its core operations on a single thread. DAG planning, task scheduling, result collection, and Python driver code all execute sequentially on the main driver thread. Additional CPU cores on the driver are used for background JVM garbage collection, monitoring threads, and network I/O handling but the primary scheduling and coordination workload is not parallelised. A driver with 8 cores will show low utilisation for most Spark workloads because the critical path is single-threaded. The exception is workloads with heavy Python multiprocessing or multiple concurrent Spark contexts.

THE FIX

Size the driver SKU for memory requirements rather than CPU. The driver needs memory proportional to the data it collects, the broadcast variables it manages, and the Python objects it holds. A driver with 32GB RAM and 4 cores (for example `Standard_E4ds_v5`) is appropriate for most production workloads. Reserve large multi-core driver SKUs for workloads that explicitly use Python multiprocessing or run multiple concurrent Spark queries from the same driver. Monitor actual driver memory usage rather than CPU when right-sizing.

22

DRIVER AND WORKER SIZING

You sized for peak data volume. Your cluster sits idle 90 percent of the time.

THE SITUATION

A cluster is sized to handle the maximum expected data volume which occurs during month-end processing. The cluster runs all month but month-end processing represents only 3 days of data-intensive work. For the remaining 27 days the cluster is provisioned at peak size but processing only 10 percent of peak volume.

WRONG THINKING

Engineers size for worst case to ensure the platform never fails during peak periods. Provisioning for peak and letting the cluster run at that size continuously seems safer than risking undersizing. The cost of idle peak capacity across the non-peak days is not calculated.

WHAT IS ACTUALLY HAPPENING

A cluster sized for peak volume that runs continuously pays for peak capacity even during low-volume periods. If peak processing requires 20 workers and normal processing requires 4, running 20 workers all month to handle 3 days of peak work means 27 days of paying for 16 idle workers. For a 30-day month this represents 90 percent of cluster cost delivering no processing value. On Azure both DBU and VM charges accrue. The financial inefficiency compounds when multiple clusters follow the same sizing pattern. Azure Reserved Instances do not help here because they are paid for whether used or not.

THE FIX

Design compute configuration to match actual workload patterns. For predictable peak periods use scheduled cluster resizing: run a smaller cluster during normal operations and scale up automatically before peak periods using the Databricks REST API or an Azure Data Factory orchestration. Alternatively use autoscaling with a low minimum and a high maximum so the cluster scales down during low-volume periods and scales up during peak. For month-end batch jobs, provision a dedicated peak cluster that runs only during the peak window rather than sizing the always-on cluster for peak.

04

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 4

Spot Instances

Seven scenarios covering Azure Spot VM failure modes, the specific workload patterns where Spot causes more problems than it saves, and how to build resilient compute that handles eviction gracefully.

23

SPOT INSTANCES

Your job failed halfway through. A Spot VM was evicted.

THE SITUATION

An Azure Databricks job runs for 4 hours processing a large dataset. At the 3.5-hour mark the job fails with a lost executor error. Investigation reveals an Azure Spot VM was evicted. The job must restart from the beginning because there are no checkpoints.

WRONG THINKING

Engineers enable Azure Spot VMs to reduce compute costs and assume eviction failures are rare events that can be handled by rerunning the job manually. The cost of rerunning a 4-hour job from scratch is not calculated against the Spot savings.

WHAT IS ACTUALLY HAPPENING

Azure evicts Spot VMs when on-demand capacity is needed in the region, when the spot price exceeds your configured maximum price, or during planned maintenance. Eviction rates vary by SKU, region, and time of day and are published in the Azure portal as historical percentages per SKU per region. Azure provides a 30-second eviction notice via the Scheduled Events metadata endpoint, but 30 seconds is not enough time to gracefully drain a Spark worker in the middle of a stage. When a Spot worker is evicted during a Spark job, the executor is lost and any shuffle data or cached data on that VM's local temp disk is gone. Spark must recompute the lost partitions from the last checkpoint or from the beginning of the job if no checkpoint exists. For a 4-hour job, recomputation from scratch can cost more in total compute than the Spot savings delivered.

THE FIX

Use Spot VMs with on-demand fallback for critical long-running jobs. In the Azure Databricks cluster configuration set worker nodes to Spot with on-demand fallback so that if Spot capacity is unavailable or evicted, on-demand VMs fill the gap. For jobs longer than 1 hour, implement checkpointing to Delta tables at regular intervals so recomputation starts from the last checkpoint rather than the beginning. Calculate the expected cost of eviction and recomputation before assuming Spot saves money on long-running jobs.

24

SPOT INSTANCES

You enabled Spot VMs to cut cost. Jobs now randomly fail at 9am.

THE SITUATION

After enabling Azure Spot VMs across all job clusters to reduce costs, jobs that previously ran reliably begin failing intermittently during business hours. Some runs complete successfully, others fail with lost executor errors. Failures cluster between 9am and 12pm local time and are difficult to reproduce in overnight testing.

WRONG THINKING

Engineers attribute the random failures to infrastructure instability or Azure Databricks bugs. Spot VM eviction as a cause is not immediately considered because Spot was working without incident during overnight testing. The difference in Azure capacity demand between business hours and overnight is not known.

WHAT IS ACTUALLY HAPPENING

Azure capacity pressure on popular SKUs peaks during local business hours when most workloads in the region are running. Eviction rates for general-purpose SKUs like `Standard_DS4_v2` and `Standard_D8s_v5` in popular regions like East US, West Europe, and Southeast Asia rise significantly between 9am and 12pm local time as production workloads start their daily runs and consume on-demand capacity. Overnight testing in those regions sees lower eviction rates because demand is lower. This creates an intermittent failure pattern that appears random but is driven by predictable Azure capacity demand curves visible in the Azure Spot eviction rate dashboard.

THE FIX

Monitor Spot eviction rates by time of day using cluster event logs. Use the Azure portal's eviction rate data for your target SKU and region to identify whether the SKU has high eviction during business hours. If business-hours eviction rates exceed 5 percent for the target SKU, switch business-hours critical jobs to on-demand or use Spot with on-demand fallback configured aggressively. Reserve aggressive Spot usage for overnight batch jobs where eviction rates are typically lower.

25

SPOT INSTANCES

Your Spot VM was evicted during a shuffle. The entire job restarted from scratch.

THE SITUATION

A Spot worker is evicted during a large shuffle stage in a 2-hour job. Spark detects the lost executor, marks the shuffle outputs from that node as unavailable, and requires the preceding stages to be recomputed to rebuild the shuffle data. The job effectively restarts from the beginning and takes another 2 hours to complete.

WRONG THINKING

Engineers assume Spark is resilient to node failures and will retry only the failed tasks rather than rerunning the entire job. Spark's fault tolerance mechanisms are assumed to handle Spot eviction transparently with minimal impact.

WHAT IS ACTUALLY HAPPENING

Spark is resilient to task failures but not to the loss of shuffle data. During a shuffle, each executor writes its output partitions to local disk on the VM. These shuffle files are required by the downstream stage. If an executor is lost and the VM is deallocated, the local temp disk and the shuffle files on it are gone. Spark cannot proceed with the downstream stage because the required input data no longer exists. Spark must recompute all the stages that produced that shuffle data, effectively rerunning everything up to the lost shuffle stage. For a job where the shuffle is halfway through, this means rerunning half the job. For some configurations it means rerunning the entire job.

THE FIX

For long-running jobs that involve large shuffles, prefer serverless compute which uses disaggregated shuffle storage decoupled from worker VMs, so worker eviction does not destroy shuffle data. On classic compute, structure long jobs as a sequence of Delta table writes and reads to ADLS Gen2 rather than a single large job so that intermediate results are persisted and recomputation starts from the last successful write rather than the beginning. Do not run shuffle-heavy stages on aggressive Spot configurations without on-demand fallback.

26

SPOT INSTANCES

You set Spot max price to the on-demand price. You are still not getting Spot capacity.

THE SITUATION

A data engineer configures Azure Spot VMs with the maximum price set to the on-demand price to maximise the chance of acquiring Spot capacity. The cluster repeatedly falls back to on-demand instances. Spot VMs are never actually provisioned. The cloud cost is the same as running on-demand.

WRONG THINKING

Engineers assume that setting the maximum price equal to on-demand guarantees Spot acquisition because the price ceiling is competitive. When Spot is not provisioned despite a high maximum price, the cause is assumed to be a Databricks configuration issue.

WHAT IS ACTUALLY HAPPENING

Azure Spot does not use a bidding model in the traditional sense. The maximum price you configure is an eviction trigger: if the Spot price rises above your max price, your VM is evicted. Setting max price equal to on-demand means your VMs will not be evicted on price (only on capacity), but it does not guarantee allocation. Azure Spot allocation depends on whether the region has spare capacity for your chosen SKU at the moment of allocation. When demand for the SKU is high in the target region, Spot capacity may simply not be available regardless of max price. The Azure portal's eviction rate data and the SKU availability dashboard reveal this pattern.

THE FIX

Switch to on-demand when Spot capacity for the required SKU is consistently unavailable in the target region. Use the Azure portal's Spot VM advisor and SKU availability tools to identify SKUs with high Spot availability before selecting them for Spot workloads. Consider a different SKU with better Spot availability in the same region (E-series often has different availability than D-series). For Azure Databricks, configure instance pools with multiple compatible SKUs so the cluster can fall back to whichever SKU has capacity at the moment of provisioning.

27

SPOT INSTANCES

Your on-demand fallback is not triggering. The job simply fails instead.

THE SITUATION

A cluster is configured with Spot workers and on-demand fallback. When a Spot VM is evicted mid-job, the expected fallback to on-demand does not occur. The job fails with a lost executor error instead of replacing the Spot VM with an on-demand one.

WRONG THINKING

Engineers assume the on-demand fallback configuration works automatically in all failure scenarios. Having set it up once, it is trusted to handle Spot eviction transparently. The specific conditions under which fallback triggers versus does not trigger are not investigated.

WHAT IS ACTUALLY HAPPENING

On-demand fallback in Azure Databricks cluster configurations primarily applies at cluster provisioning time, not at runtime eviction. When the cluster is being created, if Spot capacity is unavailable the cluster can fall back to on-demand VMs for initial provisioning. However, when a Spot VM is evicted after the cluster is already running, Azure Databricks attempts to replace it with another Spot VM rather than switching to on-demand. If no Spot capacity is available at the moment of replacement, the executor slot may remain unfilled and the job may fail depending on the cluster's minimum worker configuration and the job's tolerance for executor loss.

THE FIX

Configure cluster policies to use a hybrid mix: set the on-demand base count to a minimum number of workers that can sustain the job even if all Spot workers are lost, and allow additional capacity above that as Spot. The on-demand base ensures the job can complete at reduced speed even without any Spot. For jobs where executor loss cannot be tolerated, run on on-demand VMs entirely and apply Spot only to development and non-critical batch workloads.

28

SPOT INSTANCES

Spot evictions are rare in dev. Constant in production at peak hours.

THE SITUATION

A team uses Azure Spot VMs in both development and production environments. Development runs without any Spot evictions over several months. Production jobs, which run during business hours, experience frequent Spot evictions particularly between 9am and 12pm.

WRONG THINKING

Engineers validate Spot reliability in the development environment and assume the same reliability in production. The difference in SKU choice, region, and time of day between development and production environments is not considered.

WHAT IS ACTUALLY HAPPENING

Development clusters typically use smaller SKUs, run in off-peak hours, and may run in a different Azure region with lower overall demand. Production clusters often use larger, more popular SKUs in primary business regions like East US 2 or West Europe. Between 9am and 12pm local time in those regions, demand for large compute SKUs spikes as thousands of production workloads start their daily batch runs simultaneously. Spot availability drops and eviction rates increase precisely when production jobs are most active. The Spot market conditions that make development reliable are fundamentally different from production peak-hour conditions in busy regions.

THE FIX

Evaluate Spot suitability separately for development and production environments rather than assuming they behave the same. For production jobs running during business hours on popular SKUs, use the Azure Spot eviction rate data for the specific SKU and region. If eviction rates are above 5 percent for your target SKU during business hours, on-demand compute will typically deliver better total cost when recomputation time is factored in. Reserve Spot for overnight batch jobs and development workloads where eviction rates are lower.

29

SPOT INSTANCES

You ran your streaming driver on Spot. A single eviction broke your checkpoint.

THE SITUATION

A Structured Streaming job runs on a cluster where both driver and workers use Azure Spot VMs to reduce cost. A Spot eviction targets the driver VM. The streaming job fails and on restart it cannot read the checkpoint cleanly because the driver was killed mid-write to the checkpoint directory.

WRONG THINKING

Engineers use Spot VMs for streaming to reduce the cost of continuously running clusters. Since streaming jobs are long-running, Spot savings appear significant. The specific vulnerability of streaming checkpoints to driver eviction is not understood, and the cluster configuration that put the driver on Spot was not reviewed.

WHAT IS ACTUALLY HAPPENING

In Structured Streaming, checkpoint integrity depends on the driver writing checkpoint metadata in a transactional sequence to the checkpoint directory (typically on ADLS Gen2 via abfss). The driver writes checkpoint files to mark completed micro-batches and update the state store. If the driver is killed mid-write due to a Spot eviction with only 30 seconds notice, the checkpoint directory may be left in an incomplete state for the in-flight batch. On restart, Spark typically recovers cleanly from the last committed batch on Delta-backed sinks, but recovery is slower and in some configurations may require manual intervention to clear the in-flight batch metadata. Worker eviction is recoverable through task retry; driver eviction is far more disruptive.

THE FIX

Never run the Spark driver on Spot VMs for streaming workloads. In the Azure Databricks cluster configuration, the default is on-demand driver with optional Spot workers, but verify this is the case. Set the driver SKU to on-demand explicitly. Workers can remain Spot to capture cost savings on the majority of compute while protecting the streaming driver from eviction-induced disruption. Use Delta Lake as the streaming sink so checkpoint and output state share a transactional store.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 5

Cluster Policies

Six scenarios covering cluster policy design, governance failures that happen when policies are too loose or too tight, and the specific gaps between what a policy enforces and what engineers assume it enforces.

A data scientist spun up a 100-node cluster. Nobody noticed for a week.

THE SITUATION

A data scientist in a shared Azure Databricks workspace creates a 100-node cluster to run a large machine learning training job. The cluster runs continuously for a week before a finance review identifies the cost. By that point it has consumed several thousand dollars of DBUs and Azure VM charges, and the data scientist has finished the work and forgotten to terminate it.

WRONG THINKING

Workspace administrators assume that data scientists understand cost implications and will manage their own clusters responsibly. Governance is informal and relies on team communication rather than technical controls. Cost monitoring is reviewed monthly in the Azure billing portal rather than in real time.

WHAT IS ACTUALLY HAPPENING

Without cluster policies restricting maximum cluster size, any user with cluster-create permission in an Azure Databricks workspace can create a cluster of any size their workspace permits. A 100-node cluster started by a single user can consume more capacity in a day than the entire team consumes in a week under normal operations. Without real-time cost alerts, large clusters can run unnoticed for extended periods. The cost visibility in Azure Databricks and Azure Cost Management requires active monitoring and does not automatically alert administrators when unusual cluster configurations are created.

THE FIX

Implement cluster policies that restrict maximum cluster size for different user groups. Data scientists should have policies limiting clusters to a maximum of 10 to 20 workers for interactive development. Large training jobs should require a different policy with approval or should be submitted as jobs with time limits. Set up Azure Cost Management budget alerts at the resource group or subscription level that trigger when daily or weekly spend exceeds a threshold. Configure Databricks budget policies for workspace-level spend tracking. Set auto-termination on all clusters to a maximum of 2 to 4 hours of inactivity so forgotten clusters do not run indefinitely.

31

CLUSTER POLICIES

Your cluster policy is set. Users are creating clusters that bypass it.

THE SITUATION

A cluster policy is configured restricting users to specific VM SKUs and maximum worker counts. Despite the policy being active, users are creating clusters with unrestricted configurations. Investigation reveals the clusters bypassing the policy are using a different policy called Unrestricted that was present in the workspace before the governance policy was created.

WRONG THINKING

Workspace administrators create a governance policy and assume it applies to all users. The existence of the default Unrestricted policy is not considered. Since the governance policy is active and assigned to the user group, administrators assume it is the only policy available to those users.

WHAT IS ACTUALLY HAPPENING

Azure Databricks workspaces include a built-in Unrestricted policy that allows cluster creation with any configuration. Users who have been granted Can Use access to multiple policies can choose which policy to apply when creating a cluster. If the Unrestricted policy is available to users alongside the governance policy, users can select Unrestricted when they want to avoid the governance constraints. The governance policy only applies when users choose to use it. Simply creating a restrictive policy does not prevent users from using the Unrestricted policy if they have access to it.

THE FIX

Remove Can Use access to the Unrestricted policy for all user groups that should be governed by cluster policies. In the Azure Databricks workspace admin settings, explicitly set the Unrestricted policy to admin-only access. Verify that the governance policy is the only policy available to non-admin users (and non-admin Entra ID groups). After removing Unrestricted access, test cluster creation as a non-admin user to confirm only the governed policy appears in the cluster creation dialog.

32

CLUSTER POLICIES

You restricted cluster creation. Now nobody can run jobs without admin help.

THE SITUATION

After a cost governance incident, workspace administrators restrict cluster creation permissions to admin-only. Job failures start appearing immediately because automated jobs that previously created their own job clusters can no longer do so. Data engineers cannot run ad-hoc notebooks. Every cluster creation now requires an admin ticket and the team productivity drops significantly.

WRONG THINKING

Administrators respond to a governance problem by removing cluster creation permissions entirely. The downstream impact on automated jobs and engineering workflows is not evaluated before the change is made. Governance is implemented as a binary restriction rather than a policy-based control.

WHAT IS ACTUALLY HAPPENING

Azure Databricks jobs that use job cluster compute create a new cluster for each run as part of the job execution. If the service principal (typically an Entra ID service principal or managed identity registered with the workspace) or user running the job does not have cluster creation permissions, the job fails immediately when it attempts to provision the cluster. Removing cluster creation from all non-admin principals also removes it from the service principals used for automation, breaking all jobs that rely on job clusters. The fix for ungoverned cluster creation is not to remove creation permissions but to constrain what can be created through policies.

THE FIX

Restore cluster creation permissions to users and service principals but implement cluster policies that constrain the allowed configurations. Use separate policies for interactive development, job compute, and ML workloads with different size limits and allowed VM SKUs for each. Assign the appropriate policy to each user group or Entra ID security group rather than removing permissions entirely. This preserves the ability to create clusters while enforcing governance through policy constraints.

33

CLUSTER POLICIES

Your cluster policy allows autoscaling to 50 nodes. The default is 50.

THE SITUATION

A cluster policy is created with an autoscaling maximum of 50 nodes. The intent is to allow burst capacity up to 50 nodes when needed. In practice, every cluster created under this policy autoscales to 50 nodes almost immediately and stays there. Costs are significantly higher than expected.

WRONG THINKING

Administrators set the policy maximum to 50 as a ceiling, assuming users will configure their clusters to a sensible maximum within that limit. The policy maximum is interpreted as a hard cap rather than a default. Users creating clusters under this policy inherit 50 as the default maximum autoscaling value rather than setting their own.

WHAT IS ACTUALLY HAPPENING

Cluster policy values work as either fixed values, default values, or allowed ranges depending on how they are configured in the policy JSON. If the autoscaling maximum is set as a default value rather than a restriction, users creating clusters under the policy see 50 as the pre-filled maximum and may not change it. An autoscaling cluster with a maximum of 50 will scale to 50 nodes whenever it encounters a workload that queues tasks, which includes most non-trivial Spark jobs. The policy did not prevent high-cost clusters from being created because the maximum was a default, not an enforced limit appropriate for the workload.

THE FIX

Review each cluster policy value and distinguish between values that should be fixed, values that should be capped with a maximum, and values that are defaults users can change. For autoscaling maximums, set a reasonable fixed or maximum value that reflects the actual workload requirements rather than a theoretical ceiling. Separate policies by workload type so interactive development has a low autoscaling maximum of 5 to 10 nodes and dedicated batch jobs have higher limits only when the workload justifies it.

34

CLUSTER POLICIES

You set an auto-termination policy. Long-running notebooks keep resetting the timer.

THE SITUATION

A cluster policy sets auto-termination to 60 minutes of inactivity. The intent is to shut down clusters that are left running after engineers finish work. In practice clusters are running for 8 to 12 hours because notebooks are sending keep-alive commands that reset the inactivity timer, even when the engineer is not actively working.

WRONG THINKING

Administrators set auto-termination expecting it to terminate clusters after 60 minutes of actual idle time. The definition of inactivity in Databricks is not understood. Notebooks that appear idle from the user's perspective are assumed to be idle from the cluster's perspective.

WHAT IS ACTUALLY HAPPENING

Databricks measures cluster inactivity as the absence of submitted Spark commands. An attached notebook that is not running any cells does not reset the inactivity timer. However, some notebook features such as background variable explorer updates, auto-complete queries, and notebook health checks can submit lightweight Spark commands that reset the timer. A notebook attached to a cluster with these features active may prevent auto-termination from firing even when the engineer has left for the day. Additionally, any long-running command that exceeds the inactivity timeout will keep the cluster alive until it completes.

THE FIX

Set auto-termination to a value that reflects acceptable idle cost rather than expected session length. For development clusters a 30 to 60-minute auto-termination is appropriate and background keep-alives will occasionally extend this but will not prevent termination indefinitely once the engineer's session is fully inactive. For tighter cost control, supplement auto-termination with a maximum lifetime policy that terminates clusters after an absolute duration regardless of activity. Maximum lifetime of 8 to 10 hours ensures overnight clusters are terminated even if background activity prevents inactivity-based termination.

35

CLUSTER POLICIES

Your cluster policy enforces VM SKUs. Users found a workaround using pools.

THE SITUATION

A cluster policy restricts VM SKUs to cost-effective `Standard_D`-series options. The policy is working as expected for clusters created directly. However, users have discovered they can create instance pools with any SKU including expensive GPU and memory-optimised instances, and then create clusters that attach to those pools. The cluster policy SKU restriction does not apply to pool-backed clusters.

WRONG THINKING

Administrators assume the cluster policy covers all compute creation paths including instance pools. Governance is validated by testing direct cluster creation and confirming the policy restrictions are enforced. The interaction between cluster policies and instance pool configurations is not tested.

WHAT IS ACTUALLY HAPPENING

Cluster policies in Azure Databricks govern the configuration of the cluster itself including VM SKU when the cluster provisions its own VMs directly. When a cluster is backed by an instance pool, the SKU is determined by the pool configuration rather than the cluster configuration. A cluster policy that restricts SKUs does not propagate that restriction to the pool the cluster attaches to. Users who have permission to create instance pools can create a pool with any SKU and then create a policy-compliant cluster that attaches to that pool, effectively bypassing the SKU restriction.

THE FIX

Extend governance to cover instance pool creation in addition to cluster creation. Create pool policies that restrict SKUs in the same way cluster policies do. Restrict instance pool creation permissions to a controlled set of approved pool configurations that match your governance requirements. Review existing pools in the workspace and terminate any that use SKUs outside the approved list. Test the complete compute creation path including pool-backed clusters when validating governance policies.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 6

Instance Pools

Seven scenarios covering how instance pools work on Azure, the specific ways they fail to deliver the startup time improvements they promise, and the hidden Azure VM costs they create when misconfigured.

36

INSTANCE POOLS

You created an instance pool. Clusters using it still have cold start delays.

THE SITUATION

An instance pool is created with a minimum idle count of 5 to eliminate job cluster cold starts. Clusters are configured to use the pool. Job runs still show 3 to 5 minute startup delays, nearly identical to before the pool was created.

WRONG THINKING

Engineers assume the instance pool will completely eliminate cluster startup time. Since VMs are pre-warmed in the pool, attaching a cluster to the pool should start instantly. When startup delays persist, the pool configuration is assumed to be broken.

WHAT IS ACTUALLY HAPPENING

Instance pools pre-warm Azure VMs, eliminating VM allocation time. However cluster startup involves additional steps beyond VM allocation: installing the Databricks runtime image on the pooled VMs if it is not already cached, configuring the Spark session, running init scripts (often pulled from DBFS or ADLS Gen2), and installing cluster libraries. If the cluster has init scripts or libraries not cached on the pool VMs, these installation steps run after the VM is acquired from the pool and still take significant time. The pool eliminates the VM allocation step but not the software configuration step.

THE FIX

Minimise init scripts and library installations on clusters backed by instance pools to get the full cold start benefit. Move library installations to the pool level using preloaded Databricks Runtime versions so they are pre-installed on pool VMs before clusters attach. Keep init scripts on pool-backed clusters minimal and avoid heavy package installations. For clusters that require complex library setups, accept that startup will take longer and set SLAs accordingly. Benchmark startup time with and without init scripts to quantify the contribution of each component to total startup time.

37

INSTANCE POOLS

Your instance pool has 10 idle instances. Your job still waits for a cluster.

THE SITUATION

An instance pool has 10 idle VMs available. A job cluster configured to use this pool shows a multi-minute wait before the cluster reaches Running state. The pool idle count confirms VMs are available. The job is not acquiring them.

WRONG THINKING

Engineers assume a cluster configured to use a pool will immediately acquire idle VMs from it. With 10 idle VMs available the cluster should start instantly. When it does not, the pool is assumed to be misconfigured or broken.

WHAT IS ACTUALLY HAPPENING

Instance pools are configured with a specific VM SKU. A cluster can only acquire VMs from a pool if the cluster's driver and worker SKUs match the SKU configured in the pool. If the cluster specifies a SKU different from what the pool contains, the cluster cannot use the pool's idle VMs and must fall back to allocating new VMs from Azure Compute. This mismatch is common when cluster configurations are created independently of pool configurations or when cluster policies allow SKU selection that does not match available pools.

THE FIX

Verify that the cluster's driver and worker SKUs exactly match the SKU configured in the pool. In the cluster creation UI, pools that are compatible with the selected SKU appear at the top of the pool dropdown. If the cluster SKU does not match the pool, no pools will appear as compatible. Either align the cluster SKU to match the pool or update the pool to use the required SKU. Use cluster policies to enforce that pool-backed clusters use only SKUs available in the designated pools.

38

INSTANCE POOLS

Instance pool costs are higher than just running the cluster directly.

THE SITUATION

An instance pool is created with a minimum idle count of 10 to ensure fast cluster startup. After one month the cost of the pool including idle VM charges is higher than the previous month when clusters were created on demand without a pool.

WRONG THINKING

Engineers assume instance pools reduce cost by making clusters faster and more efficient. The cost of keeping idle VMs in the pool is not calculated before setting the minimum idle count. Pre-warming is assumed to be a cost-neutral or cost-saving feature.

WHAT IS ACTUALLY HAPPENING

Instance pools charge for idle VMs at the Azure VM rate even when no cluster is using them. The DBU charge applies only when a cluster is actively attached to a pool VM and running Spark workloads. However, the underlying Azure VM cost accrues continuously for every idle instance in the pool and shows up on the Azure consumption bill independently of the Databricks marketplace charge. A pool with 10 minimum idle instances runs 10 VMs continuously at Azure pay-as-you-go pricing even during hours when no jobs are running. For workloads that run for only a few hours per day, the cost of 24-hour idle VM reservation often exceeds the cost of provisioning job clusters on demand.

THE FIX

Set pool minimum idle count based on actual cluster demand patterns rather than a fixed number. If jobs run for 4 hours per day, a minimum idle count of 0 with a non-zero maximum idle count for post-job retention may be more cost-effective than keeping 10 VMs idle 24 hours a day. Calculate the break-even point: idle VM cost per hour times idle hours must be less than the startup latency cost of provisioning new VMs. For workloads with low daily run frequency, on-demand cluster provisioning without a pool is often cheaper than pool maintenance costs. Consider Azure Reserved Instances for the underlying pool VMs if the minimum idle count is steady.

39

INSTANCE POOLS

You shared a pool across teams. One team consumed all instances at peak time.

THE SITUATION

A shared instance pool serves multiple teams in an Azure Databricks workspace. During a morning peak period one team runs a large batch job that acquires all available pool VMs. Other teams attempting to start clusters at the same time find no pool VMs available and their clusters either wait or fail to start.

WRONG THINKING

Pool administrators assume shared pools work fairly across teams and that the pool's maximum capacity provides a buffer for all users. The possibility of one team monopolising all pool VMs is not anticipated in the pool design.

WHAT IS ACTUALLY HAPPENING

Instance pools have a maximum capacity that applies to all users of the pool collectively. There is no per-team or per-user quota within a single pool. A team that submits a job requiring 20 workers will acquire up to 20 VMs from the pool regardless of whether other teams need VMs simultaneously. If the pool maximum is 20 and one team acquires all 20, other teams receive zero VMs and must wait for the pool to replenish from Azure VM allocation, which reintroduces the cold start delay the pool was designed to eliminate.

THE FIX

Create separate pools for different teams or workload types rather than sharing a single pool. Each team's pool has an independent maximum capacity that prevents one team from exhausting the shared resource. Alternatively create pools with team-specific SKUs so different teams use different pools naturally. If a shared pool is necessary, implement cluster policies that limit the maximum cluster size for each team to a fraction of the pool capacity, ensuring no single team can acquire all available VMs.

40

INSTANCE POOLS

Your pool instances are idle but the cluster cannot attach to them.

THE SITUATION

An instance pool shows idle VMs available. A cluster configured to use the pool fails to attach and shows an error indicating no instances are available from the pool. The pool idle count is not zero.

WRONG THINKING

Engineers see idle VMs in the pool and assume the cluster should be able to use them. When attachment fails despite available VMs the error message is assumed to be incorrect or the pool is assumed to have a bug.

WHAT IS ACTUALLY HAPPENING

Instance pools enforce several constraints beyond SKU matching. A pool has a maximum number of VMs that can be attached to running clusters at any one time. If this maximum is reached, the pool will not provide additional VMs even if idle instances are shown in the pool metrics. Additionally, idle instance counts can reflect a momentary snapshot and actual availability may differ at the moment of cluster creation if other clusters are simultaneously acquiring VMs. Pool-level access control via Entra ID groups can also prevent specific users or service principals from acquiring VMs from the pool even when capacity is available.

THE FIX

Check the pool's attached instance count against its maximum capacity in the pool configuration. If attached instances equal the maximum, the pool is at capacity regardless of the idle count displayed. Increase the pool maximum capacity or reduce attached cluster sizes to free up pool capacity. Verify that the user or service principal creating the cluster has Can Attach To permission on the pool. Review pool events in the pool detail page to identify the specific reason for attachment failure.

41

INSTANCE POOLS

You set pool min idle to 5. Azure VM charges appear even when no jobs run.

THE SITUATION

An instance pool is configured with a minimum idle count of 5. The team expects compute charges only when jobs are actively running. At month end the Azure consumption report includes VM charges for hours when no jobs were running. The idle pool VMs appear to be generating cost continuously.

WRONG THINKING

Engineers conflate DBU charges and VM charges and assume both apply only when Spark is running. An idle VM sitting in a pool waiting for a cluster to attach is not running any Spark workloads, so it is assumed to be free. The Azure billing model for idle pool VMs is not checked before setting a minimum idle count.

WHAT IS ACTUALLY HAPPENING

Azure Databricks does not charge DBUs for idle pool VMs. However, the underlying Azure VM cost accrues continuously at the pay-as-you-go rate for the VM SKU, regardless of whether a cluster is attached. These VM charges appear on the Azure consumption report under the Databricks managed resource group, separate from the Databricks marketplace charge. A pool with 5 minimum idle `Standard_D8s_v5` VMs running 24 hours a day generates continuous Azure VM cost. At low usage rates, idle pool VM costs can exceed the cost of the actual job runs the pool is designed to accelerate.

THE FIX

Set pool minimum idle count to zero for workloads that run infrequently. With a minimum of zero the pool releases all VMs when no clusters are attached and incurs no idle VM charges. Clusters attaching to a zero-minimum pool will not get pre-warmed VMs but they will not generate idle costs during off-hours. Set a non-zero minimum only for pools serving frequent or time-sensitive jobs where the startup latency benefit justifies the idle VM cost. Schedule a pool resize to increase the minimum idle count before known peak periods and decrease it afterward. Consider Azure Reserved Instances or a Savings Plan for VMs in pools with a steady non-zero minimum.

42

INSTANCE POOLS

Your pool is in the right region but your storage is in another. Egress dominates job cost.

THE SITUATION

An Azure Databricks workspace and its instance pool are in West Europe. The ADLS Gen2 storage account holding the source data is in North Europe. Cluster startup is fast but Spark job performance is significantly degraded with slow network I/O. The Azure bill shows large data egress charges.

WRONG THINKING

Engineers focus on cluster startup time as the primary metric for pool effectiveness. Job performance after startup is not benchmarked against the non-pool baseline. The geographic relationship between the workspace, pool, and storage is not checked when the pool is configured.

WHAT IS ACTUALLY HAPPENING

Azure Databricks instance pools must be in the same Azure region as the workspace that uses them; cross-region pool usage is not supported. Within a region, the pool inherits the availability zone configuration of the workspace. However, the workspace may legitimately exist in a different region from the ADLS Gen2 storage account holding the data. Cross-region traffic between compute and storage in Azure incurs both higher latency and per-GB egress charges that dominate job performance and cost for data-heavy workloads. A correctly configured pool does not fix this; it simply reveals the storage locality problem more clearly because the compute is fast and reliable.

THE FIX

Verify that the Azure Databricks workspace, its instance pools, and the primary ADLS Gen2 storage accounts are all in the same Azure region. Use the Azure portal to confirm region alignment. If the data is in a different region from compute, the egress cost and latency of cross-region data access will dominate job performance regardless of cluster startup optimisation. Co-locate all components of an Azure Databricks deployment in the same region. For workspaces that need access to data in multiple regions, evaluate Azure Data Factory or storage replication patterns to bring data into the compute region rather than pulling cross-region at query time.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 1

Chapter 7

Runtime and Configuration

Eight scenarios covering Databricks Runtime version behavior on Azure, Spark configuration pitfalls, and the silent failures that appear when runtime assumptions do not match the actual execution environment.

43

RUNTIME AND CONFIGURATION

You upgraded the Databricks Runtime. Your job broke with no code changes.

THE SITUATION

A Databricks Runtime upgrade from DBR 13 to DBR 14 is applied to a job cluster. The job code is unchanged. After the upgrade the job fails with a type mismatch error that did not exist on the previous runtime. Rolling back to DBR 13 makes the job work again.

WRONG THINKING

Engineers assume Databricks Runtime upgrades are backward compatible and that code running on an older runtime will run identically on a newer one. A runtime upgrade is treated as a maintenance operation that requires no application-level validation.

WHAT IS ACTUALLY HAPPENING

Databricks Runtime upgrades include new versions of Apache Spark, Delta Lake, and bundled libraries. Each major and minor Spark version can introduce behavioral changes including stricter type checking, changed default configurations, modified function behavior, and removed deprecated APIs. A join or cast operation that Spark 3.3 resolved implicitly may fail in Spark 3.5 due to stricter type enforcement. A function that returned a nullable result in an older runtime may return a non-null result in a newer one, changing downstream logic. Runtime upgrades are not minor infrastructure changes. They are Spark and library version upgrades that require application-level testing.

THE FIX

Treat every Databricks Runtime upgrade as a software release requiring regression testing. Maintain a staging Azure Databricks workspace running the new runtime version and run all production jobs against it before upgrading production clusters. Pin critical production jobs to a specific runtime version using the cluster configuration and upgrade deliberately rather than automatically. Review the Databricks Runtime release notes for the target version to identify breaking changes in Spark, Delta Lake, and bundled libraries that may affect your workloads.

44

RUNTIME AND CONFIGURATION

Your Spark config is set on the cluster. It is being ignored at runtime.

THE SITUATION

A Spark configuration property is set in the cluster's Spark configuration section. The job runs but behaves as if the configuration is not applied. Setting the same property in the notebook using `spark.conf.set()` at the start of the session makes it work correctly.

WRONG THINKING

Engineers assume Spark configurations set at the cluster level apply to all sessions running on that cluster. The cluster configuration is the authoritative source for all Spark properties and notebook-level settings are only needed for ad-hoc overrides.

WHAT IS ACTUALLY HAPPENING

Not all Spark configuration properties can be set at cluster creation time. Azure Databricks classifies configuration properties into static properties set at cluster start and dynamic properties that can be changed within a session. Some properties must be set before the `SparkContext` is created and cannot be changed after the session starts. Other properties are session-level and must be set within the session to take effect. If a property that requires session-level application is set only at the cluster level, Spark may silently ignore it or apply a default value. Additionally some properties are blocked by Azure Databricks for security or stability reasons (particularly anything that would override workspace security settings) and will appear to be set but have no effect.

THE FIX

Verify whether the configuration property you are trying to set is static or dynamic in the Databricks documentation. Static properties set at cluster creation apply to all sessions. Dynamic properties require `spark.conf.set()` within the session to take effect. For dynamic properties that must apply to all job runs, set them in the notebook or script at the beginning of execution. Check the Databricks documentation for blocked or overridden properties that cannot be changed from their platform defaults.

45

RUNTIME AND CONFIGURATION

You set `spark.sql.shuffle.partitions` to 200. Your job has 3 partitions of data.

THE SITUATION

A job is configured with `spark.sql.shuffle.partitions` set to 200 following a best practice recommendation. The source data has 3 partitions. After every shuffle operation the job produces 200 output partitions, most of which contain zero or very few rows. Downstream operations are slow and the job creates hundreds of tiny output files on ADLS Gen2.

WRONG THINKING

Engineers set shuffle partitions to 200 based on a widely cited default recommendation. The recommendation is applied as a universal setting without considering the actual data volume, the partition count of the specific workload, or whether Adaptive Query Execution is enabled.

WHAT IS ACTUALLY HAPPENING

The shuffle partition count determines how many output partitions Spark creates after a shuffle operation such as a join or aggregation. Setting it to 200 with 3 input partitions means Spark attempts to distribute data across 200 output partitions regardless of how little data there is. Without AQE, each empty partition still represents a task that must be scheduled, executed, and completed, and produces a separate output file when written. The result is 200 tiny files, 200 task scheduling overheads, and significantly worse performance than running with 3 or 6 shuffle partitions. Small files are particularly costly on ADLS Gen2 because each file requires a separate HTTP request to list and read, and the per-request overhead dominates for files under 1MB.

THE FIX

The modern best practice on Azure Databricks is to enable Adaptive Query Execution (the default on all current DBR versions) by ensuring `spark.sql.adaptive.enabled` is true and `spark.sql.adaptive.coalescePartitions.enabled` is true. With AQE, set `spark.sql.shuffle.partitions` to a relatively high value (200 to 800 depending on cluster size) and let AQE automatically coalesce small partitions into larger ones at runtime based on actual data distribution. AQE eliminates the empty-partition problem without requiring per-job tuning. For workloads where AQE is not viable, set shuffle partitions to roughly 2 to 3 times the total executor cores.

46

RUNTIME AND CONFIGURATION

Your ML Runtime cluster is running ETL jobs. You are paying for GPU you never use.

THE SITUATION

A cluster is configured with the Databricks ML Runtime and a GPU-enabled VM SKU like `Standard_NC6s_v3`. The cluster was originally provisioned for machine learning model training but is now used primarily for ETL pipeline runs. The GPU is never utilised during ETL work.

WRONG THINKING

Engineers provision an ML Runtime cluster for flexibility so it can handle both ML training and ETL tasks without switching clusters. Using one cluster for both purposes seems simpler than maintaining separate clusters. The cost premium of GPU SKUs and ML Runtime DBUs for non-GPU workloads is not calculated.

WHAT IS ACTUALLY HAPPENING

GPU-enabled SKUs on Azure cost significantly more than CPU-only SKUs, often 3 to 10 times more depending on the GPU family (NC, ND, NV). The Databricks ML Runtime adds additional DBU charges on top of standard DBU rates. For ETL workloads that do not use GPU acceleration, a GPU SKU is pure cost overhead. The GPU sits idle during all ETL processing. Spark ETL jobs use the CPU and memory of the VM, not the GPU, so there is no performance benefit from the GPU either. Running ETL on a GPU ML Runtime cluster pays premium pricing for hardware and software that provides zero value for the workload, on both the Azure VM line and the Databricks DBU line.

THE FIX

Separate ML training workloads from ETL workloads onto appropriately sized and typed clusters. Use CPU-only SKUs (D, E, or F series) with a standard Databricks Runtime for ETL pipelines. Reserve ML Runtime and GPU SKUs for workloads that explicitly use GPU-accelerated libraries such as model training with PyTorch or TensorFlow. Calculate the cost differential between a GPU ML Runtime cluster and a CPU standard cluster running the same ETL workload. The savings from right-typing the cluster are typically substantial.

47

RUNTIME AND CONFIGURATION

You pinned your cluster to an old runtime. Security patches stopped applying.

THE SITUATION

A production cluster is pinned to DBR 11.3 LTS because the team experienced a breaking change when upgrading and does not want to deal with another upgrade. Six months later a security audit reveals that several CVEs affecting Databricks Runtime components have been patched in newer runtime versions but not in 11.3 LTS which has reached end of support.

WRONG THINKING

Engineers pin to an LTS runtime version for stability and assume LTS means the runtime will receive ongoing security patches indefinitely. Stability and security are treated as the same thing. Staying on an older version is perceived as safer than upgrading.

WHAT IS ACTUALLY HAPPENING

Databricks LTS runtimes receive security patches and bug fixes for a defined support window, currently three years from release per the published LTS lifecycle. After the support window ends, no further patches are applied to that runtime version even for critical security vulnerabilities. A cluster pinned to an end-of-support runtime is running software with known unpatched CVEs in Spark, the JVM, Python packages, and system libraries bundled in the runtime. For regulated environments on Azure (financial services, healthcare) this can be a compliance violation surfaced during Azure Policy audits or Microsoft Defender for Cloud scans. The perception that older is more stable is accurate for functional behavior but incorrect for security posture.

THE FIX

Review the Databricks Runtime support lifecycle documentation to identify the end-of-support date for every pinned runtime version in your workspace. For production workloads, maintain a rolling upgrade process that moves clusters to a current LTS runtime before their current version reaches end of support. Establish a testing process for runtime upgrades using a staging Azure Databricks workspace so breaking changes are identified and resolved before production clusters are upgraded. Never run production clusters on end-of-support runtimes regardless of functional stability.

48

RUNTIME AND CONFIGURATION

Your job runs fine on DBR 12. Fails silently on DBR 14 with wrong results.

THE SITUATION

A job is migrated from a cluster running DBR 12 to a cluster running DBR 14. The job completes successfully with no errors. Downstream data validation fails because aggregated values differ from expected. Rolling back to DBR 12 produces correct results. The job produces wrong results on DBR 14 without raising any error or warning.

WRONG THINKING

Engineers validate a runtime upgrade by checking for errors and job completion status. A job that completes without errors is assumed to be producing correct results. Silent result differences are not anticipated as a consequence of a runtime upgrade.

WHAT IS ACTUALLY HAPPENING

Spark version upgrades between major versions can change the behavior of certain functions, type casting rules, and null handling in ways that do not raise errors but produce different results. A division operation that returned integer results in an older Spark version may return floating point results in a newer one. A window function with a specific frame boundary may have subtle ordering differences in edge cases. An implicit cast that succeeded silently in one version may apply a different conversion rule in another. These behavioral changes are often documented in release notes but are easy to miss when upgrading multiple versions at once.

THE FIX

Implement data validation as part of every runtime upgrade process. After running the job on the new runtime, compare output counts, aggregated values, and key metrics against a reference run on the previous runtime. Any discrepancy indicates a behavioral change that must be investigated. Do not rely on job completion status alone to validate a runtime upgrade. For jobs with complex aggregations or type casting, review the Spark migration guide for all versions between the current and target runtime before upgrading.

49

RUNTIME AND CONFIGURATION

You set executor memory to 80 percent. Garbage collection is now killing performance.

THE SITUATION

A Spark job is experiencing spill to disk and a data engineer increases `spark.executor.memory` to allocate 80 percent of the available VM memory to the executor JVM. Spill is eliminated. However job duration increases significantly and the Spark UI shows long GC pause times consuming 30 to 40 percent of task execution time.

WRONG THINKING

Engineers see spill to disk as the primary performance problem and increase executor memory to fix it. Eliminating spill is treated as success. The secondary effect of large JVM heaps on garbage collection behavior is not anticipated.

WHAT IS ACTUALLY HAPPENING

JVM garbage collection performance degrades non-linearly as heap size increases. A JVM with an 8GB heap can run short GC pauses that are barely noticeable. The same JVM with a 48GB heap may run GC pauses lasting several seconds because the GC must scan a much larger memory space to identify unreachable objects. G1GC and other modern GC algorithms mitigate this at moderate heap sizes but very large heaps still produce longer pauses. When GC pauses consume 30 to 40 percent of task time, adding more memory to eliminate spill can make overall performance worse even though it solved the specific spill problem.

THE FIX

Balance spill elimination against GC overhead when tuning executor memory. Instead of allocating the maximum possible memory to each executor, consider using more executors with smaller heaps by choosing VM SKUs with more cores and less memory per core, for example `Standard_F16s_v2` instead of a single very large E-series VM. Each executor has an independent heap and GC cycle, so 4 executors with 12GB heaps will typically outperform 1 executor with 48GB when GC overhead is significant. Use the Spark UI GC time column in the executor metrics to quantify GC overhead before and after memory changes. Aim for GC time below 10 percent of total task time.

50

RUNTIME AND CONFIGURATION

You set executor memory to 80 percent of VM RAM. The cluster keeps losing executors.

THE SITUATION

A cluster is configured with executor memory set to 80 percent of each worker VM's advertised RAM. The remaining 20 percent was assumed to be sufficient for the operating system and Databricks agent processes. During heavy processing jobs, worker nodes begin failing with out of memory errors at the OS level and executors are killed by the Linux OOM killer.

WRONG THINKING

Engineers allocate 80 percent of VM memory to Spark executors and assume 20 percent is sufficient headroom for system processes. The 80-20 split seems conservative enough. The memory requirements of Databricks agent processes, the Azure Linux Agent, monitoring daemons, and OS overhead are not quantified before setting this ratio.

WHAT IS ACTUALLY HAPPENING

Worker VMs in Azure Databricks run several processes in addition to the Spark executor JVM. The Databricks agent monitors the cluster and reports metrics, the Azure Linux Agent (waagent) handles VM lifecycle and extensions, the operating system kernel and system services consume memory, log shipping agents write executor logs to ADLS Gen2 or the configured destination, and Python processes for PySpark serialization run alongside the JVM. On memory-intensive workloads, the JVM also exceeds its configured heap allocation temporarily due to off-heap memory usage for shuffle buffers and native libraries. Additionally, Azure VMs reserve memory for the hypervisor, so the advertised RAM is not all usable by the guest OS to begin with. When the total memory consumption of all processes exceeds physical RAM available to the guest, the Linux OOM killer terminates the process consuming the most memory, which is typically the Spark executor JVM. The node then loses its executor and Spark must recompute the lost tasks.

THE FIX

Set executor memory to a maximum of 70 percent of advertised VM RAM rather than 80 percent, leaving 30 percent for hypervisor reservation, Azure Linux Agent, Databricks agent, and OS overhead on Databricks worker nodes. For VMs with large total RAM such as 128GB or 256GB, a smaller percentage is needed because absolute system overhead does not scale linearly with VM size. Monitor actual memory utilisation on worker nodes using the cluster metrics view and reduce executor memory allocation if nodes are consistently near 100 percent total memory usage. Use memory-optimised SKUs (E-series) with larger absolute RAM to provide more headroom rather than pushing the executor memory percentage higher.

CONCLUSION

Compute problems look like code problems until you know where to look.

Across 50 scenarios the same pattern repeats. The cluster is blamed last. Engineers optimise queries, rewrite transformations, and add workers before they check whether the cluster type, VM SKU, or Azure-side configuration is the actual bottleneck. A job that fails because the driver has too little memory is not a query problem. A job that runs slowly because autoscaling fired too late is not a Spark problem. A job that costs too much because the instance pool keeps idle VMs running 24 hours a day is not a compute problem at all, it is a billing model misunderstanding. Recognising which layer the problem lives in is the first step to fixing it quickly.

ALSO IN THE SERIES

Delta Lake

- You ran OPTIMIZE. Your queries got slower.
- Your MERGE is creating duplicate rows despite a unique match condition.

Workflows and Orchestration

- Your workflow job shows success. One task silently produced no output.
- Your DLT expectations are failing. The pipeline succeeds anyway.

And more. Download the full series free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is Book I in the Azure Databricks Engineering Patterns series. Every scenario is set in an Azure Databricks context: VM SKUs, Azure Spot eviction model, VNet injection, secure cluster connectivity, ADLS Gen2, Entra ID service principals, and the dual Azure VM and DBU billing model. The patterns and fixes reflect the platform as it stands today. Databricks and Azure both evolve quickly and some guidance may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome. Reach out through any of the channels below.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT

-  www.ssanjaychandra.com
-  linkedin.com/in/ssanjaychandra
-  ssanjaychandra@outlook.com